

**UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO**  
**FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**  
**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS**



**DOCUMENTO DE ESPECIFICACIÓN Y DISEÑO ARQUITECTÓNICO (V2)**

*Resolución de Ejercicios Ágiles, Requerimientos IEEE 830, Modelado UML y Base de Datos Relacional*

---

**“SISTEMA INTEGRAL DE GESTIÓN DE CAPACITACIONES (SIGC-CUSCO)”**

**Incremento Iterativo – Versión V2.0**

---

**Asignatura:** INGENIERÍA DE SOFTWARE I

**Docente:** Ing. Lisha Sabah Diaz Caceres

**Enfoque:** Desarrollo Incremental e Iterativo – Ingeniería de Requisitos

**Presentado por:**

- |                                    |        |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Yaranga Achahui, Aldo            | 103179 |
| • Ccama Enriquez, Carolay          | 210921 |

# Índice

|                                                                                           |           |
|-------------------------------------------------------------------------------------------|-----------|
| <b>1. Resolución Formal de Ejercicios Prácticos de Flujo Ágil</b>                         | <b>1</b>  |
| 1.1. Ejercicio 1: Una Revisión Saturada (Gestión de Flujo y Cuellos de Botella) . . . . . | 1         |
| 1.2. Ejercicio 2: Coordinar una Entrega (Objetivo de Ciclo y Roles) . . . . .             | 3         |
| <b>2. Especificación Formal de Requerimientos de Software</b>                             | <b>4</b>  |
| 2.1. Requerimientos Funcionales (RF) Priorizados bajo MoSCoW . . . . .                    | 5         |
| 2.2. Requerimientos No Funcionales (RNF) según ISO/IEC 25010 . . . . .                    | 6         |
| <b>3. Plan de Entrega del Sprint 3: Inscripción con DNI y Credencial QR</b>               | <b>6</b>  |
| 3.1. Historias de Usuario y Criterios de Aceptación . . . . .                             | 6         |
| 3.2. Definition of Ready y Definition of Done . . . . .                                   | 8         |
| 3.3. Ley de Little con Números del Tablero . . . . .                                      | 8         |
| 3.4. Métricas de Flujo a Medir en el Tablero . . . . .                                    | 9         |
| 3.5. Acuerdo de Trabajo con Git . . . . .                                                 | 9         |
| <b>4. Modelado Visual del Sistema (UML)</b>                                               | <b>9</b>  |
| 4.1. Diagrama de Casos de Uso del Sistema (UML) . . . . .                                 | 9         |
| 4.2. Diagrama de Clases del Dominio y Multiplicidades . . . . .                           | 10        |
| <b>5. Diseño de Base de Datos Relacional y Esquema Físico</b>                             | <b>11</b> |
| 5.1. Modelo Relacional y Reglas de Integridad Referencial . . . . .                       | 11        |
| 5.2. Script DDL de Implementación Física (PostgreSQL) . . . . .                           | 12        |
| <b>6. Arquitectura de Software y Stack Tecnológico del Proyecto</b>                       | <b>13</b> |
| 6.1. Capas de la Arquitectura de Software . . . . .                                       | 13        |
| 6.2. Resumen de Tecnologías y Entorno de Desarrollo . . . . .                             | 14        |
| <b>7. Plan de Evolución Incremental e Iterativo por Sprints</b>                           | <b>15</b> |

# 1. Resolución Formal de Ejercicios Prácticos de Flujo Ágil

A continuación, se presentan las soluciones técnicas y metodológicas a los dos escenarios prácticos planteados en clase, aplicadas al contexto del proyecto **SIGC-CUSCO**. Cada ejercicio cierra con el entregable solicitado: el primero con los límites *WIP* y una política de bloqueo; el segundo con el objetivo de ciclo, las responsabilidades y los criterios de calidad.

## 1.1. Ejercicio 1: Una Revisión Saturada (Gestión de Flujo y Cuellos de Botella)

**Escenario Planteado:** El límite de la columna *Revisión (Review)* es 1 y ya existe un ítem esperando.

**Estado del tablero en el escenario** (véase la Figura 1): *Review / QA* tiene 1 de 1 tarjetas (US-02, en revisión activa); *In Progress* tiene 2 de 2 (US-03 bloqueada y US-06 terminada, esperando cupo en *Review*). Es decir, no existe ningún cupo libre aguas abajo.

### 1. ¿Conviene iniciar la tarea de recordatorios? Justificación:

- **Decisión técnica:** NO conviene iniciarla.
- **Fundamento en la Ley de Little:** en un sistema de flujo se cumple

$$WIP = TH \times CT \implies CT = \frac{WIP}{TH} \quad (1)$$

donde *WIP* es el trabajo en curso, *TH* es el rendimiento (*Throughput*) y *CT* es el tiempo de ciclo (*Cycle Time*). Con *Review* saturada, el rendimiento *TH* del equipo está limitado por esa columna. Abrir el módulo de recordatorios elevaría el *WIP* sin aumentar *TH*, de modo que *CT* crecería para todas las tarjetas, incluidas las que ya esperan revisión.

- **Fundamento de negocio:** los recordatorios no forman parte del objetivo del siguiente ciclo (inscripción con DNI y credencial QR) y no son un requerimiento crítico del sistema. No hay costo de oportunidad en posponerlos; sí lo hay en dejar trabajo terminado sin validar.
- **Principio rector:** “*Stop starting, start finishing*” (dejar de empezar y enfocarse en terminar).

### 2. Acuerdo de colaboración para terminar la revisión (*Swarming*):

- **Noe Choquenaira** (autor de US-06, que está esperando cupo) se suma a la revisión de US-02 junto con el revisor asignado, **Aldo Yaranga**: ejecutan juntos las pruebas de aceptación y resuelven las observaciones en el momento, en lugar de abrir una tarea nueva.
- **Carolay Ccama** (facilitadora) prepara la lista de criterios de aceptación de US-02 y cronometra la sesión. Si la revisión supera una hora, se decide en ese momento si se aprueba con observaciones menores o se devuelve a *In Progress*.
- Al aprobarse US-02, pasa a *Done* y libera el cupo de *Review*. US-06 entra de inmediato, y la capacidad que queda libre se usa para revisarla, no para iniciar trabajo nuevo.

### 3. Política de bloqueo: qué se hace si una tarjeta queda bloqueada:

- a) **Marcar de inmediato.** Quien detecta el bloqueo agrega la etiqueta negra BLOQUEADA y escribe en la tarjeta la causa, la dependencia externa (por ejemplo, “*Pendiente token de la API de RENIEC para validar DNI*”), quién puede destrabarla y la fecha y hora en que se bloqueó.

- b) **Sigue contando en el WIP.** La tarjeta permanece en su columna y ocupa su cupo. Así el costo del bloqueo se mantiene visible y nadie abre trabajo nuevo para esquivarlo.
- c) **Tratar en el siguiente *Daily Standup*.** La facilitadora la pone primera en la agenda (los primeros 5 minutos) y se acuerda una acción concreta con responsable.
- d) **Plan alternativo y escalamiento a las 24 horas.** Si sigue bloqueada después de 24 horas, se escala al docente o al responsable de la dependencia y se activa un plan alternativo (por ejemplo, validar el DNI por formato de 8 dígitos y unicidad hasta que llegue el token).
- e) **Mientras tanto, nadie queda ocioso.** La persona asignada a la tarjeta bloqueada apoya por *swarming* a otra tarjeta (revisión o pruebas). No inicia elementos nuevos.
- f) **Cierre del bloqueo.** Al resolverse se retira la etiqueta y se anota en la tarjeta cuánto duró el bloqueo, para analizar las causas recurrentes en la retrospectiva.

| Columna     | Límite WIP | Justificación                                                                                                                      |
|-------------|------------|------------------------------------------------------------------------------------------------------------------------------------|
| Backlog     | Sin límite | Lista priorizada por el Product Owner; no es trabajo en curso.                                                                     |
| To Do       | 5          | Reserva de trabajo listo para tomar, equivalente a un ciclo. Un número mayor genera especificaciones que envejecen sin ejecutarse. |
| In Progress | 2          | Equipo de 3 integrantes: siempre queda una persona libre para revisar, facilitar o apoyar por <i>swarming</i> .                    |
| Review / QA | 1          | Un solo revisor activo a la vez. Obliga a terminar la revisión antes de aceptar más trabajo terminado.                             |
| Done        | Sin límite | Solo ingresan tarjetas que cumplen la Definition of Done (Ejercicio 2).                                                            |

Tabla 1: Límites WIP del tablero SIGC-CUSCO y su justificación.

#### Entregable del Ejercicio 1: WIP y Política de Bloqueo

- **Límites WIP:** *To Do* = 5 | *In Progress* = 2 | *Review / QA* = 1 | *Backlog* y *Done* = sin límite.
- **Regla ante saturación:** si *Review* está llena, no se inicia trabajo nuevo; se ayuda a terminar la revisión.
- **Política de bloqueo:** etiqueta BLOQUEADA con causa, la tarjeta sigue contando en el WIP, se trata en el siguiente *Daily Standup*, se escala a las 24 horas y mientras tanto se apoya a otra tarjeta.

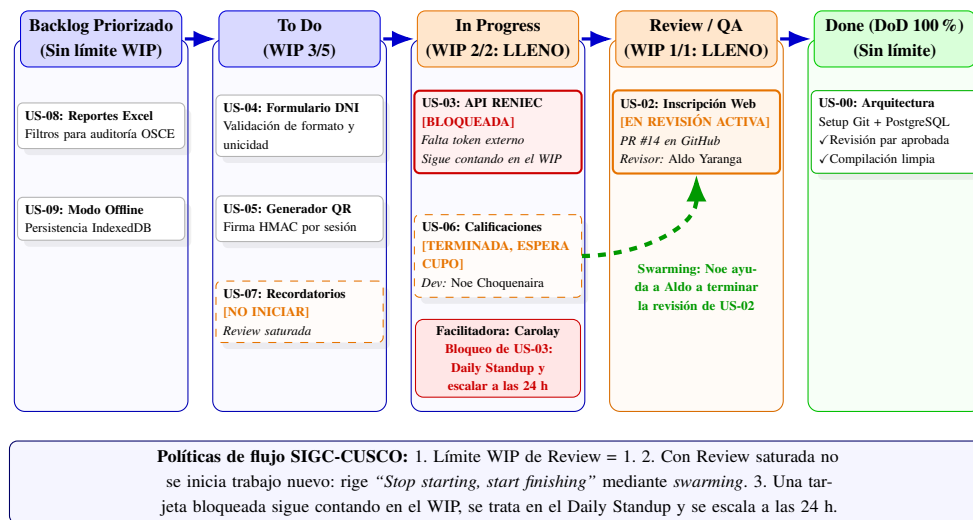


Figura 1: Flujo Kanban de SIGC-CUSCO con la columna Review saturada, el acuerdo de *swarming* y la política de bloqueo.

## 1.2. Ejercicio 2: Coordinar una Entrega (Objetivo de Ciclo y Roles)

**Escenario Planteado:** *El equipo debe mostrar una función utilizable en una entrega incremental.*

### 1. Objetivo de ciclo orientado al valor (*Sprint Goal* del Sprint 3):

*“Que un participante en Cusco pueda inscribirse en línea a un curso municipal o turístico validando su DNI, y obtenga en pantalla su credencial digital con código QR para el control de asistencia.”*

Es un objetivo, no una lista de tareas: describe lo que una persona podrá hacer al terminar el ciclo (un corte vertical completo, de la pantalla a la base de datos) y no el trabajo técnico horizontal, como crear tablas o diseñar vistas. Las tareas se derivan del objetivo y no al revés.

*Dependencia con el Ejercicio 1:* si el token de RENIEC sigue bloqueado, el objetivo se cumple con el plan alternativo (validación del DNI por formato y unicidad), de modo que un bloqueo externo no impide mostrar la función.

### 2. Responsabilidades: quién prioriza, quién facilita y quién construye:

| Pregunta         | Rol y responsable                                         | Responsabilidades concretas                                                                                                                                                               |
|------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prioriza</b>  | Product Owner: Aldo Yaranga (103179)                      | Ordena el backlog según las necesidades de municipalidades y empresas turísticas, define los criterios de aceptación y decide si una tarjeta pasa a <i>Done</i> tras ver la demostración. |
| <b>Facilita</b>  | Scrum Master y facilitadora: Carolay Ccama (210921)       | Vigila los límites WIP, dirige el <i>Daily Standup</i> , gestiona las tarjetas bloqueadas (política del Ejercicio 1) y remueve impedimentos externos.                                     |
| <b>Construye</b> | Tech Lead y desarrollo: Noe Franklin Choquenaira (133962) | Diseña la arquitectura y el esquema en PostgreSQL, construye los servicios y la validación del DNI, y responde por las pruebas y la calidad técnica del incremento.                       |

Tabla 2: Roles del equipo SIGC-CUSCO para el ciclo de entrega.

En un equipo de tres personas, todos construyen y revisan; el rol define quién decide y quién responde en cada aspecto. Se aplica la regla *autor*  $\neq$  *revisor*: ningún integrante aprueba su propio trabajo.

### 3. Dos evidencias para considerar el trabajo hecho (*Definition of Done*):

- **Evidencia 1, demostración funcional en vivo:** en el navegador se ingresa un DNI, el sistema valida que haya vacantes, guarda la inscripción y muestra de inmediato el código QR. La valida el Product Owner frente al criterio de aceptación.
- **Evidencia 2, calidad del software:** *Pull Request* en GitHub aprobado por los dos integrantes que no son autores, con 0 advertencias del *linter* y el script SQL ejecutado sin errores de integridad. La verifica la facilitadora, que deja registrado el enlace del *Pull Request* en la tarjeta.

#### Entregable del Ejercicio 2: Objetivo, Responsabilidades y Calidad

- **Objetivo:** inscripción en línea con validación de DNI y credencial con QR en pantalla.
- **Responsabilidades:** Aldo Yaranga prioriza y acepta, Carolay Ccama facilita el flujo y los bloqueos, Noe Choquenaira construye y responde por la calidad técnica.
- **Calidad:** una tarjeta solo pasa a *Done* con la demostración en vivo aprobada y el *Pull Request* revisado por dos integrantes, sin advertencias del *linter* y con el SQL sin errores.

## 2. Especificación Formal de Requerimientos de Software

Bajo los estándares internacionales IEEE 830 e ISO/IEC/IEEE 29148, se formaliza la especificación técnica del sistema SIGC-CUSCO.

## 2.1. Requerimientos Funcionales (RF) Priorizados bajo MoSCoW

| Código       | Nombre del Requisito       | Especificación Funcional Detallada                                                                                             | Prioridad   |
|--------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>RF-01</b> | Control de Acceso RBAC     | Autenticación segura y autorización por roles: Administrador, Coordinador Municipal/Turístico, Docente y Participante.         | Must Have   |
| <b>RF-02</b> | Gestión Multientidad       | Parametrización aislada de instituciones organizadoras (Municipalidades y Empresas de Turismo) con logos oficiales.            | Must Have   |
| <b>RF-03</b> | Catálogo de Capacitaciones | Creación de cursos con definición de fechas, horas pedagógicas, cupo máximo, docente asignado y temario.                       | Must Have   |
| <b>RF-04</b> | Inscripción con DNI        | Formulario público con validación algorítmica de DNI peruano, verificación de unicidad y control automático de vacantes.       | Must Have   |
| <b>RF-05</b> | Asistencia Digital QR      | Registro instantáneo de asistencia mediante escaneo del código QR del participante desde el dispositivo móvil del docente.     | Must Have   |
| <b>RF-06</b> | Calificaciones y Actas     | Registro de notas en escala vigesimal (0 a 20), cálculo automático de condición de aprobación y cierre inmutable de actas.     | Should Have |
| <b>RF-07</b> | Emisión de Certificados    | Generación masiva de certificados PDF firmados digitalmente, provistos de código QR y hash de autenticidad SHA-256.            | Must Have   |
| <b>RF-08</b> | Verificador Público Web    | Portal web público donde cualquier entidad puede escanear el QR o ingresar el código para validar la autenticidad del diploma. | Must Have   |
| <b>RF-09</b> | Modo Offline de Asistencia | La interfaz del docente permite registrar asistencias sin conectividad a internet, sincronizando datos al recuperar enlace.    | Should Have |
| <b>RF-10</b> | Reportes de Fiscalización  | Generación de padrones de asistencia y notas exportables a formato Excel (.xlsx) compatibles con auditorías de OSCE.           | Could Have  |

Tabla 3: Matriz de Requerimientos Funcionales con priorización MoSCoW.

## 2.2. Requerimientos No Funcionales (RNF) según ISO/IEC 25010

| Código | Dimensión de Calidad             | Criterio de Aceptación Cuantitativo y Tecnológico                                                                                                                 |
|--------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RNF-01 | Seguridad<br>Criptográfica       | Integridad documental mediante hash SHA-256 inmutable por certificado.<br>Almacenamiento de credenciales con cifrado adaptativo <i>bcrypt</i> ( $cost \geq 12$ ). |
| RNF-02 | Rendimiento en Lote              | Generación asíncrona de 100 certificados PDF en un tiempo total menor a 15 segundos sin degradar el tiempo de respuesta del servidor web ( $< 500$ ms).           |
| RNF-03 | Disponibilidad del Servicio      | Arquitectura desplegada en nube con disponibilidad operativa mínima del 99.5 % en horarios hábiles de oficina.                                                    |
| RNF-04 | Usabilidad Móvil<br>(Responsive) | Interfaz adaptable que permite completar la inscripción ciudadana en máximo 3 pantallas y menos de 60 segundos en dispositivos móviles.                           |
| RNF-05 | Tolerancia a Desconexión         | Persistencia local de registros de asistencia mediante <i>IndexedDB</i> en el navegador para tolerar cortes de red en locales comunales de Cusco.                 |

Tabla 4: Requerimientos No Funcionales clasificados según el estándar ISO/IEC 25010.

## 3. Plan de Entrega del Sprint 3: Inscripción con DNI y Credencial QR

Esta sección convierte el objetivo de ciclo del Ejercicio 2 en un plan ejecutable: historias de usuario con criterios de aceptación medibles, condiciones de entrada y de salida, métricas de flujo y el acuerdo de trabajo con Git.

### 3.1. Historias de Usuario y Criterios de Aceptación

Cada historia se deriva de un requerimiento de la Tabla 3 y se acepta solo si sus criterios se cumplen en la demostración. Los puntos son una estimación inicial que el equipo confirma en la planificación del ciclo.

| ID    | Historia y criterios de aceptación                                                                                                                                                                                                                                                                                                                                                                                                                                           | Requisito        | Pts | Responsable        |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|-----|--------------------|
| US-10 | <i>Como participante, quiero ver los cursos con vacantes para elegir en cuál inscribirme.</i><br><i>Dado</i> que existen cursos abiertos, <i>cuando</i> entro al portal, <i>entonces</i> veo nombre, fecha, modalidad y vacantes de cada uno.                                                                                                                                                                                                                                | RF-03            | 3   | Noe<br>Choquenaira |
| US-04 | <i>Como participante, quiero inscribirme con mi DNI para reservar mi vacante.</i><br><i>Dado</i> un curso con vacantes, <i>cuando</i> ingreso un DNI válido de 8 dígitos y mis datos, <i>entonces</i> se guarda la inscripción.<br><i>Dado</i> un DNI ya inscrito en ese curso, <i>cuando</i> intento inscribirme de nuevo, <i>entonces</i> el sistema lo rechaza con un mensaje claro.<br>El proceso dura menos de 60 segundos y usa como máximo 3 pantallas en un celular. | RF-04,<br>RNF-04 | 5   | Noe<br>Choquenaira |
| US-02 | <i>Como coordinador, quiero que el sistema controle las vacantes para no sobrepasar el cupo.</i><br><i>Dado</i> un curso sin vacantes, <i>cuando</i> alguien intenta inscribirse, <i>entonces</i> ve el aviso “sin vacantes” y no se crea ninguna inscripción.<br><i>Dada</i> una inscripción exitosa, <i>entonces</i> las vacantes del curso bajan en 1.                                                                                                                    | RF-03,<br>RF-04  | 5   | Noe<br>Choquenaira |
| US-05 | <i>Como participante, quiero ver mi credencial con QR para usarla al registrar mi asistencia.</i><br><i>Dado</i> que terminé mi inscripción, <i>cuando</i> se guarda el registro, <i>entonces</i> veo en pantalla mi credencial con un código QR único.<br><i>Dado</i> un QR escaneado, <i>entonces</i> identifica a un solo participante y a un solo curso.                                                                                                                 | RF-05            | 5   | Noe<br>Choquenaira |
| US-03 | <i>Como coordinador, quiero contrastar el DNI con RENIEC para reducir errores de identidad.</i><br>Prioridad <i>Could Have</i> : queda fuera del objetivo mínimo del ciclo.<br>Mientras no se obtenga el token, rige la validación por formato y unicidad de US-04.                                                                                                                                                                                                          | RF-04            | 3   | Pendiente          |

Tabla 5: Sprint Backlog del Sprint 3: 18 puntos comprometidos (US-10, US-04, US-02, US-05) y 3 puntos opcionales (US-03).

Aldo Yaranga (Product Owner) acepta cada historia y Carolay Ccama (facilitadora) verifica que se respeten los límites WIP. Como todas las historias tienen a Noe Choquenaira como responsable principal, el resto del equipo participa en las revisiones y en el *swarming*; así el Ejercicio 1 se aplica de forma real y no solo en el papel.

### 3.2. Definition of Ready y Definition of Done

| Definition of Ready (para entrar a <i>To Do</i> )                                           | Definition of Done (para entrar a <i>Done</i> )                                       |
|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| La historia tiene criterios de aceptación escritos en formato <i>Dado/Cuando/Entonces</i> . | Evidencia 1: demostración funcional en vivo, validada por el Product Owner.           |
| Está vinculada a un requisito (RF o RNF).                                                   | Evidencia 2: <i>Pull Request</i> aprobado por los dos integrantes que no son autores. |
| Tiene una estimación en puntos y cabe en un solo ciclo.                                     | El <i>linter</i> no reporta advertencias y el script SQL corre sin errores.           |
| Sus dependencias externas están identificadas (por ejemplo, el token de RENIEC).            | El enlace del <i>Pull Request</i> queda registrado en la tarjeta de Trello.           |

Tabla 6: Condiciones de entrada y de salida de una tarjeta.

### 3.3. Ley de Little con Números del Tablero

Para ver el efecto de iniciar trabajo nuevo con *Review* saturada, se usa una hipótesis didáctica: el equipo termina 0,5 tarjetas por día ( $TH = 0,5$ ), es decir, una cada dos días. Con  $CT = WIP/TH$ :

| Escenario                                       | WIP activo | Tiempo de ciclo   | Lectura                                                                        |
|-------------------------------------------------|------------|-------------------|--------------------------------------------------------------------------------|
| Respetar los límites (In Progress 2 + Review 1) | 3 tarjetas | $3/0,5 = 6$ días  | Escenario del tablero.                                                         |
| Iniciar recordatorios (una tarjeta más)         | 4 tarjetas | $4/0,5 = 8$ días  | Cada tarjeta tarda 2 días más, y las que esperan revisión se retrasan igual.   |
| Dos tarjetas nuevas                             | 5 tarjetas | $5/0,5 = 10$ días | La espera sube 67 % respecto del escenario del tablero, sin entregar nada más. |
| Tres tarjetas nuevas                            | 6 tarjetas | $6/0,5 = 12$ días | La espera se duplica.                                                          |

Tabla 7: Tiempo de ciclo según el trabajo en curso, con  $TH = 0,5$  tarjetas por día (valor de referencia).

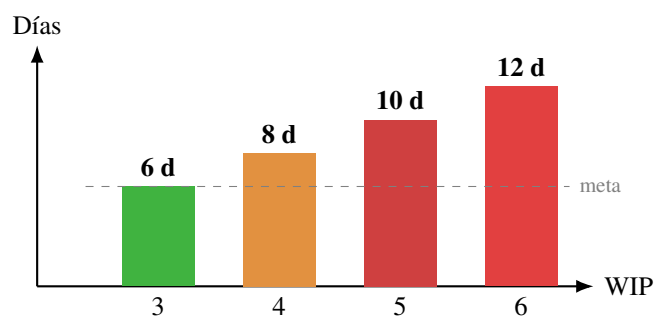


Figura 2: Tiempo de ciclo (días) al aumentar el WIP activo de 3 a 6 tarjetas, con  $TH = 0,5$ .

El throughput de referencia no es un dato medido: se reemplaza por el real al cierre del Sprint 3, leyendo las fechas de las tarjetas en Trello.

### 3.4. Métricas de Flujo a Medir en el Tablero

| Métrica                              | Cómo se mide                                                                 | Meta inicial                                           |
|--------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------|
| Tiempo de ciclo                      | Días entre que la tarjeta entra a <i>In Progress</i> y llega a <i>Done</i> . | $\leq 6$ días; alerta si supera 8.                     |
| Rendimiento<br>( <i>Throughput</i> ) | Tarjetas que llegan a <i>Done</i> por semana.                                | Medir en el Sprint 3 y fijar la meta con datos reales. |
| Edad de la tarjeta                   | Días que lleva una tarjeta en curso sin moverse.                             | Alerta si supera 2 días sin avance.                    |
| Tiempo bloqueado                     | Horas con la etiqueta BLOQUEADA.                                             | Ninguna tarjeta más de 24 horas.                       |
| Saturación de <i>Review</i>          | Porcentaje de días con la columna llena.                                     | Si supera el 50 %, revisar el proceso de revisión.     |

Tabla 8: Métricas de flujo del tablero SIGC-CUSCO. Las metas se recalibran con datos reales al cierre de cada ciclo.

### 3.5. Acuerdo de Trabajo con Git

El equipo trabaja con una rama por integrante y una rama principal *main* que solo recibe trabajo revisado.

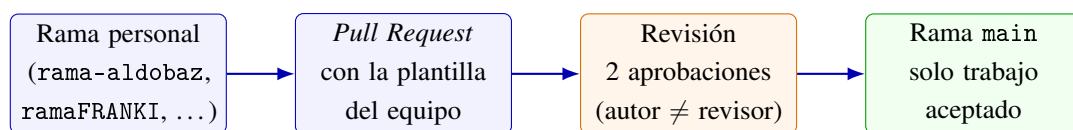


Figura 3: Flujo de integración del código y la documentación.

- **Commits:** un mensaje claro por cambio, con prefijo de tipo: `docs:`, `feat:`, `fix:` o `test:`.
- **Pull Request:** se abre con la plantilla del repositorio, que incluye la lista de la Definition of Done; nadie aprueba su propio trabajo.
- **Tarjeta y rama:** cada tarjeta de Trello indica la rama y el enlace del *Pull Request*; una tarjeta no pasa a *Done* sin ese enlace.

## 4. Modelado Visual del Sistema (UML)

### 4.1. Diagrama de Casos de Uso del Sistema (UML)

El modelo de casos de uso (Figura 4) define los límites del sistema **SIGC-CUSCO**, diferenciando los actores primarios de los servicios de auditoría y especificando las relaciones de inclusión ( $\ll \text{include} \gg$ ).

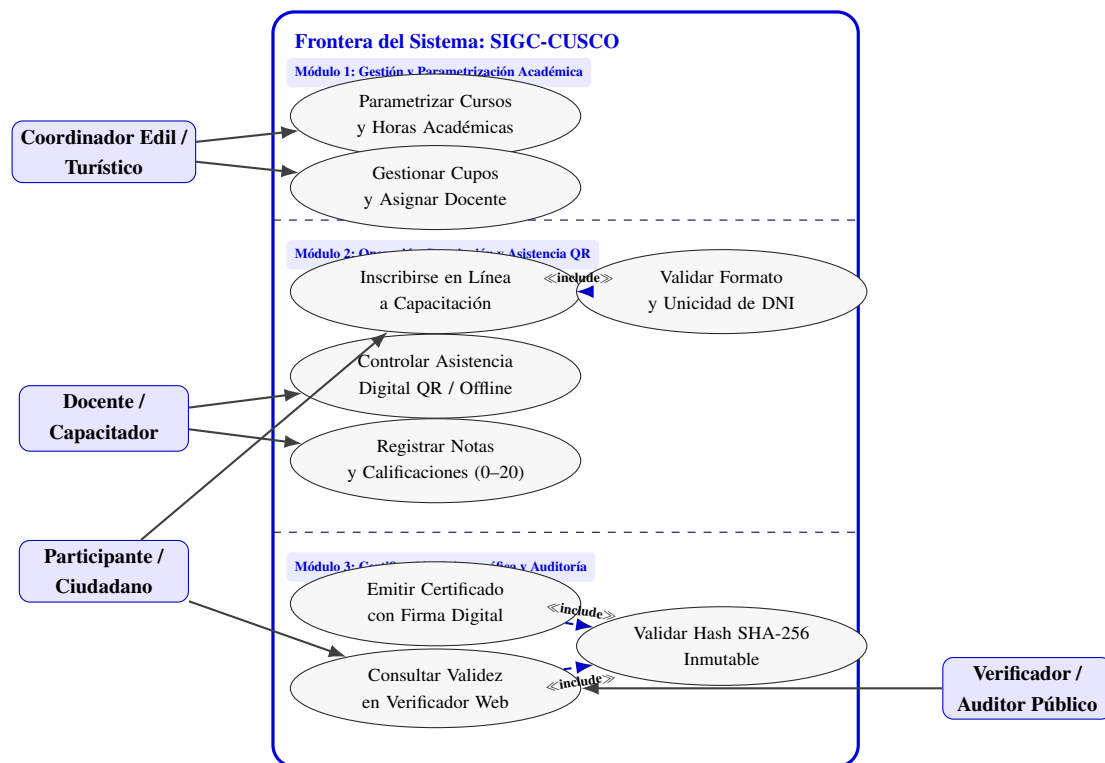


Figura 4: Diagrama de Casos de Uso del Sistema SIGC-CUSCO con módulos funcionales delimitados.

#### 4.2. Diagrama de Clases del Dominio y Multiplicidades

El modelo de clases del dominio (Figura 5) representa las entidades estructurales del negocio y sus multiplicidades formales organizadas en una topología planar sin cruces.

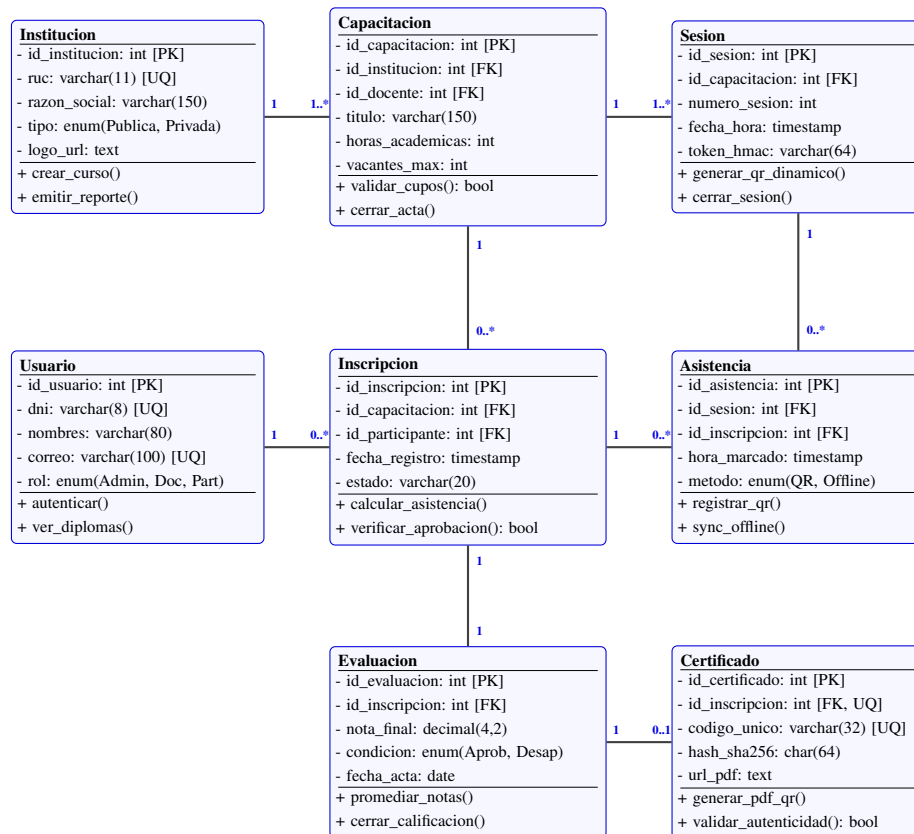


Figura 5: Diagrama de clases del dominio con 7 entidades normalizadas en grilla ortogonal, multiplicidades y trazabilidad completa.

## 5. Diseño de Base de Datos Relacional y Esquema Físico

El sistema implementa el gestor relacional **PostgreSQL 16** para asegurar soporte estricto de transacciones ACID y alta consistencia en la emisión documental.

### 5.1. Modelo Relacional y Reglas de Integridad Referencial

El diseño de datos responde a la Tercera Forma Normal (3FN), garantizando que no existan redundancias ni anomalías de actualización:

- Relación Uno a Muchos (1 : N) entre Institución y Capacitaciones:** Una entidad (municipalidad o empresa) patrocina múltiples cursos a lo largo del año. La eliminación de una institución se restringe si tiene cursos históricos cerrados (ON DELETE RESTRICT) para preservar la trazabilidad de auditoría.
- Relación Muchos a Muchos (N : M) resuelta por Entidad Asociativa:** Un participante se matricula en múltiples capacitaciones y una capacitación alberga decenas de alumnos. Esta asociación se materializa en la tabla inscripciones, la cual incluye una restricción única compuesta UNIQUE (id\_capitacion, id\_participante) para impedir duplicidades de matrícula.
- Relación Uno a Uno Condicional (1 : 0..1) con Certificados:** Una inscripción completada y aprobada genera a lo sumo un único certificado oficial. Se implementa con una restricción UNIQUE sobre la clave foránea id\_inscripcion en la tabla certificados.

## 5.2. Script DDL de Implementación Física (PostgreSQL)

### Script DDL PostgreSQL 16 – SIGC-CUSCO

```
-- 1. Entidades Organizadoras (Multitenancy)
CREATE TABLE instituciones (
    id_institucion SERIAL PRIMARY KEY,
    ruc VARCHAR(11) UNIQUE NOT NULL,
    razon_social VARCHAR(150) NOT NULL,
    tipo VARCHAR(20) NOT NULL CHECK (tipo IN ('Publica', 'Privada')),
    logo_url TEXT,
    activo BOOLEAN DEFAULT TRUE,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Usuarios del Sistema y Roles RBAC
CREATE TABLE usuarios (
    id_usuario SERIAL PRIMARY KEY,
    dni VARCHAR(8) UNIQUE NOT NULL,
    nombres VARCHAR(80) NOT NULL,
    apellidos VARCHAR(80) NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    rol VARCHAR(20) NOT NULL CHECK (rol IN ('Admin', 'Coordinador', 'Docente', 'Alumno')),
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Catálogo de Capacitaciones
CREATE TABLE capacitaciones (
    id_capitacion SERIAL PRIMARY KEY,
    id_institucion INT NOT NULL
        REFERENCES instituciones(id_institucion) ON DELETE RESTRICT,
    id_docente INT
        REFERENCES usuarios(id_usuario) ON DELETE SET NULL,
    titulo VARCHAR(150) NOT NULL,
    horas_academicas INT NOT NULL CHECK (horas_academicas > 0),
    vacantes_max INT NOT NULL CHECK (vacantes_max > 0),
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,
    estado VARCHAR(20) DEFAULT 'Planificada'
        CHECK (estado IN ('Planificada', 'En_Curso', 'Finalizada'))
);

-- 4. Matrículas e Inscripciones (Tabla Intermedia)
CREATE TABLE inscripciones (
    id_inscripcion SERIAL PRIMARY KEY,
    id_capitacion INT NOT NULL
        REFERENCES capacitaciones(id_capitacion) ON DELETE CASCADE,
    id_participante INT NOT NULL
        REFERENCES usuarios(id_usuario) ON DELETE RESTRICT,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    nota_final NUMERIC(4,2) CHECK (nota_final >= 0 AND nota_final <= 20),
    estado VARCHAR(20) DEFAULT 'Confirmada'
        CHECK (estado IN ('Confirmada', 'Aprobado', 'Desaprobado', 'Anulada')),
    CONSTRAINT uq_participante_curso UNIQUE (id_capitacion, id_participante)
);

-- 5. Control de Asistencias por Sesión (Soporte QR / Offline)
CREATE TABLE asistencias (
    id_asistencia SERIAL PRIMARY KEY,
    id_inscripcion INT NOT NULL
```

```
REFERENCES inscripciones(id_inscripcion) ON DELETE CASCADE,
numero_sesion INT NOT NULL CHECK (numero_sesion > 0),
fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
estado VARCHAR(15) NOT NULL CHECK (estado IN ('Presente', 'Tardanza', 'Falta')),
metodo VARCHAR(15) DEFAULT 'QR_Digital'
CHECK (metodo IN ('QR_Digital', 'PIN_Manual', 'Offline_Sync'))
);

-- 6. Emisión Criptográfica de Certificados
CREATE TABLE certificados (
    id_certificado SERIAL PRIMARY KEY,
    id_inscripcion INT UNIQUE NOT NULL
    REFERENCES inscripciones(id_inscripcion) ON DELETE RESTRICT,
    codigo_unico VARCHAR(32) UNIQUE NOT NULL,
    hash_sha256 CHAR(64) NOT NULL,
    fecha_emision TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    url_pdf TEXT NOT NULL
);

-- Índices de Alto Rendimiento para Búsqueda y Validación
CREATE INDEX idx_usuarios_dni ON usuarios(dni);
CREATE INDEX idx_inscripciones_curso ON inscripciones(id_capacitacion);
CREATE INDEX idx_certificados_hash ON certificados(hash_sha256);
CREATE INDEX idx_certificados_codigo ON certificados(codigo_unico);
```

## 6. Arquitectura de Software y Stack Tecnológico del Proyecto

Para garantizar robustez, mantenibilidad y escalabilidad, el proyecto **SIGC-CUSCO** adopta una arquitectura en tres capas desacopladas con componentes orientados a servicios RESTful.

### 6.1. Capas de la Arquitectura de Software

1. **Capa de Presentación (Frontend Web):** Desarrollada en **React 18** con empaquetador **Vite** y estilos utilitarios en **Tailwind CSS**. Ofrece una interfaz ligera, accesible desde teléfonos móviles de gama baja y optimizada para la interacción rápida de ciudadanos y docentes.
2. **Capa de Lógica de Negocio (Backend API):** Implementada con **Node.js** y **Express** bajo el estándar tipado de **TypeScript**. Gestiona la validación algorítmica de DNIs, el control concurrente de cupos, la orquestación del marcado de asistencia y la cola asíncrona de procesamiento de PDFs.
3. **Capa de Persistencia y Seguridad (Datos):** Motor **PostgreSQL 16** para el almacenamiento relacional estructurado. Generación de códigos QR mediante librerías nativas vectoriales y cálculo de resumen criptográfico inmutable mediante el algoritmo estándar SHA-256.

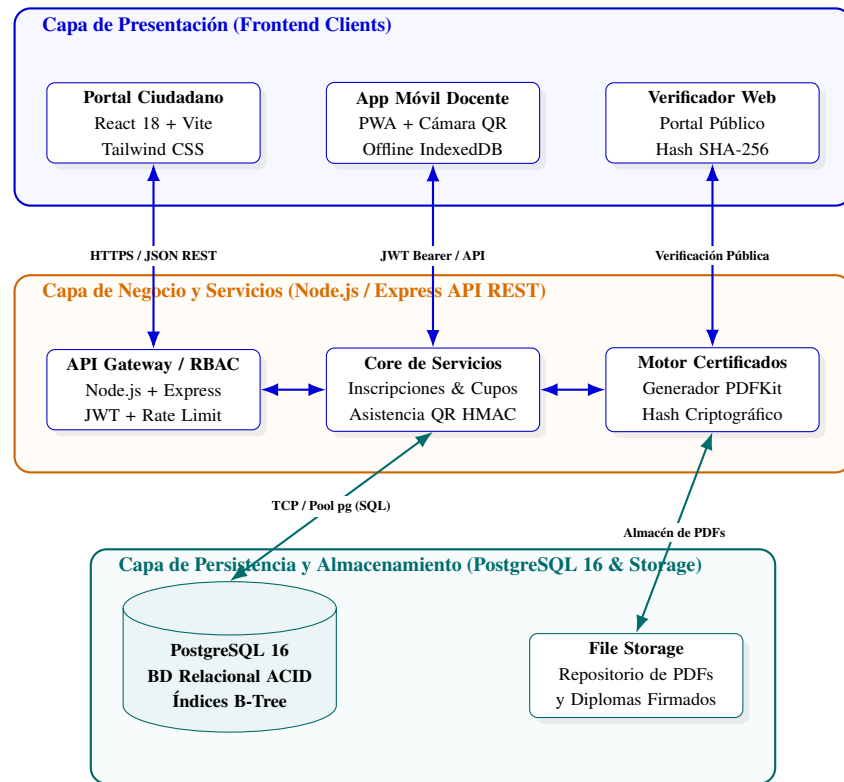


Figura 6: Arquitectura de software desacoplada en tres capas con protocolos de comunicación estandarizados.

## 6.2. Resumen de Tecnologías y Entorno de Desarrollo

| Componente / Capa    | Tecnología Elegida             | Justificación Técnica en el Proyecto                                                          |
|----------------------|--------------------------------|-----------------------------------------------------------------------------------------------|
| Base de Datos        | PostgreSQL 16                  | Soporte ACID estricto, integridad referencial y alto rendimiento en consultas indexadas.      |
| Backend / API        | Node.js + Express (TypeScript) | Asincronía eficiente ( <i>Event Loop</i> ) para procesar inscripciones masivas sin bloqueos.  |
| Frontend Web         | React + Vite + Tailwind CSS    | Renderizado rápido, diseño 100 % adaptable para smartphones y soporte de PWA.                 |
| Seguridad y Hash     | Algoritmo SHA-256 / Bcrypt     | Certificados infalsificables auditables y protección robusta de contraseñas.                  |
| Control de Versiones | Git & GitHub (SIGC-CUSCO)      | Colaboración concurrente, trazabilidad de código y despliegue continuo (CI/CD).               |
| Gestión Ágil         | Tablero Kanban en Trello       | Visualización de flujo, políticas de bloqueo y control de cuellos de botella con límites WIP. |

Tabla 9: Stack tecnológico formal adoptado para la construcción de SIGC-CUSCO.

## 7. Plan de Evolución Incremental e Iterativo por Sprints

Para cumplir con la entrega continua de valor en cada sesión del curso, el desarrollo de **SIGC-CUSCO** se planifica en cinco incrementos progresivos:

| <b>Incremento / Sprint</b>                          | <b>Entregable Técnico Principal</b>                                        | <b>Criterio de Validación Objetiva</b>                                              |
|-----------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Sprint 1: Inception</b>                          | Visión, Lean Canvas y Tablero Trello.                                      | Aprobación de alcance y matriz de stakeholders en repositorio GitHub.               |
| <b>Sprint 2: Requisitos y Arquitectura (Actual)</b> | Casos de uso, clases de dominio, DER y script DDL en PostgreSQL.           | Compilación formal del documento V2 y ejecución limpia del esquema SQL.             |
| <b>Sprint 3: Core de Inscripciones</b>              | Portal público con validación de DNI y gestión de cupos en tiempo real.    | Demostración de registro ciudadano y persistencia en base de datos.                 |
| <b>Sprint 4: Asistencia Digital</b>                 | Lector móvil de código QR con persistencia offline en IndexedDB.           | Marcado de presencia en aula comunal y sincronización con el servidor.              |
| <b>Sprint 5: Certificación y Auditoría</b>          | Motor de emisión de diplomas PDF con hash SHA-256 y validador público web. | Verificación exitosa de autenticidad escaneando el código QR con cualquier celular. |

Tabla 10: Roadmap de desarrollo incremental e iterativo del sistema SIGC-CUSCO.